PocketPulse: A Daily Expense Tracker for Android

Project Report

Submitted by: Vivek Morasiya (12302130501062)

Vandan Gabani (12302130501059)

Ajinkya Brahmane (12302130501071)

Course / Branch: CSD / Sem 7

Under the guidance of: Prof. Harsh Kantawala

College: G.H.Patel College of Engineering & Technology

Academic Year: 2026/27

Declaration

I hereby declare that the project report titled "PocketPulse: A Daily Expense Tracker for Android" is my own work, carried out under the guidance of Prof. Harsh Kantawala . It has not been submitted to any other institution for the award of any degree or diploma.

Acknowledgement

I sincerely thank my project guide, Prof. Harsh Kantawala , for the guidance and encouragement given throughout this project. I also thank the Head of the Department, the faculty of Computer Department, and my friends and family for their support.

Abstract

Managing day-to-day pocket money is a common difficulty for students and young earners. Most people only realise they have overspent when the money is already gone. PocketPulse is a lightweight Android application that addresses this by letting users record every expense as it happens, assign it to a category, and compare their spending for the day against a daily budget that they set themselves.

The application is written in Java using the native Android SDK and is built with Gradle. It follows a single-screen dashboard design that shows today's budget, amount spent, amount still available, a progress bar, a colour-coded warning message, a category-wise summary, and a list of recent transactions. All data is stored locally on the device using SharedPreferences in JSON format, so the app works fully offline and needs no account, internet connection or extra permissions. Money is stored internally in paise (as whole numbers) so that rounding errors never creep into the totals.

Keywords: Android, Java, Expense Tracker, Daily Budget, SharedPreferences, JSON, Personal Finance.

Mobile Application Development

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Technologies and Tools Used
6. System Requirements
7. System Design
8. Implementation
9. User Interface Design
10. Testing
11. Limitations
12. Future Enhancements
13. Conclusion
14. References

Appendix A: Project Structure

1. Introduction

Personal finance management has become more important as digital payments make spending quick and almost effortless. For a college student living on a fixed allowance, small daily expenses such as snacks, bus fares, stationery or a movie can add up quickly and leave little for the rest of the month.

PocketPulse (tagline: "Your money, under control.") is a mobile application designed to make expense tracking fast and simple. Instead of a complicated accounting tool, it offers one clean dashboard that answers the questions a user actually asks during the day: How much have I spent today? How much can I still spend? Where did the money go?

The project is a native Android application developed in Java. It has the package name `com.example.pocketpulse`, version 1.0, and supports Android 6.0 (API level 23) and above.

2. Problem Statement

Students and many first-time earners do not keep a record of small expenses. Paper notebooks are easily forgotten, and general-purpose finance apps are often too complex, require sign-up, show advertisements, or need internet access and permissions the user is uncomfortable giving.

There is a need for a simple, private, offline application that:

- records an expense in a few seconds,
- tells the user immediately whether they are still within their daily limit, and
- shows a clear category-wise picture of the day's spending.

3. Objectives

1. To develop an Android application that records daily expenses with a title, amount, category, optional note and automatic date and time.

2. To let the user set and change a personal daily spending limit.
3. To display the amount spent today, the amount remaining and a visual progress indicator.
4. To warn the user when spending reaches 80% of the budget, reaches the budget, or exceeds it.
5. To show a category-wise summary of today's spending.
6. To store all data locally and persistently, without requiring internet access or special permissions.
7. To validate user input so that incorrect or empty data is never saved.

4. Scope of the Project

In scope: a single-user, single-device application with daily budget tracking, six expense categories, adding and deleting expenses, a transaction history list and automatic saving of data.

Out of scope (for version 1.0): user accounts, cloud sync, multi-currency support, editing of existing expenses, monthly or yearly reports, charts and data export. Some of these are listed under Future Enhancements.

The app displays amounts in Indian Rupees (₹).

5. Technologies and Tools Used

Component	Details
Programming language	Java
Platform	Android (native, using the Android SDK)
Minimum SDK	API 23 (Android 6.0)
Target / Compile SDK	API 36
Build system	Gradle 9.5.0 with Android Gradle Plugin 9.3.3
UI definition	XML layout (`activity_main.xml`) plus views created in Java code
Data storage	SharedPreferences (key-value storage)
Data format	JSON, using the built-in `org.json` library
Money handling	`java.math.BigDecimal` and `long` values in paise
IDE	Android Studio
Version control	Git

A notable design choice is that the app uses **no third-party libraries**. It extends the standard `android.app.Activity` class and uses only classes that ship with Android, which keeps the app small and easy to build. The debug APK produced by the project is under 1 MB in size.

6. System Requirements

6.1 Hardware requirements (for running the app)

- Any Android smartphone or emulator running Android 6.0 or later
- Minimal storage (the app is under 1 MB) and standard RAM

6.2 Software requirements (for development)

- Android Studio with Android SDK Platform 36
- JDK compatible with the Gradle version used
- Gradle 9.5.0 (provided through the Gradle wrapper in the project)
- A Windows, macOS or Linux computer

6.3 Permissions

The application requests **no special permissions**. The AndroidManifest declares only the application and a single launcher activity, `MainActivity`.

7. System Design

7.1 Architecture

PocketPulse follows a simple single-activity architecture. `MainActivity` acts as the screen (view), holds the application logic, and also manages data storage. Given the small size of the app, this keeps the code easy to follow.

Layer	Responsibility	Where it lives
Presentation	Dashboard, dialogs, lists, warning messages	`activity_main.xml`, drawables, `MainActivity`
Logic	Budget calculation, validation, category totals, date checking	`MainActivity` methods
Data	Saving and loading budget and expenses	SharedPreferences (`pocketpulse_prefs`)

7.2 Data model

Each expense is represented by a private inner class, `Expense`, with the following fields:

Field	Type	Description
id	String	Unique identifier (current time in milliseconds)
title	String	Name of the expense, for example "Lunch"

Field	Type	Description
amountPaise	long	Amount in paise (1 rupee = 100 paise)
category	String	One of the six categories
note	String	Optional extra detail
timestamp	long	Date and time the expense was recorded

Why paise? Floating-point numbers such as `double` cannot represent many decimal values exactly, which can cause totals like ₹0.1 + ₹0.2 to be slightly wrong. By storing money as whole numbers of paise and converting only for display, all additions and comparisons are exact.

7.3 Categories

The app provides six fixed categories: **Food, Transport, Education, Shopping, Entertainment** and **Other**. Any unknown category read from storage is automatically mapped to "Other" by the `normalizeCategory()` method.

7.4 Storage design

Two values are kept in a SharedPreferences file named `pocketpulse_prefs`:

Key	Type	Purpose
`daily_budget_paise`	long	The user's daily budget. Default is 50000 paise (₹500).
`expenses_json`	String	A JSON array containing all saved expenses

Example of one stored expense:

```
{
  "id": "1791529457415",
  "title": "Lunch",
  "amountPaise": 12000,
  "category": "Food",
  "note": "Canteen",
  "timestamp": 1791529457415
}
```

7.5 Data flow

1. The app starts and `onCreate()` loads the layout, opens SharedPreferences, loads the budget and the expense list, and calls `refreshDashboard()`.
2. The user taps **+ Add Expense** or **Set Budget**, and a dialog opens.
3. The input is validated. If it is invalid, an error appears on the field and nothing is saved.

4. If valid, the data is updated in memory, written to SharedPreferences, and the dashboard is refreshed.
5. `refreshDashboard()` recalculates today's total, the remaining amount, the progress bar, the warning message, the category summary and the transaction list.

7.6 Budget status logic

The warning message and its colour depend on today's spending compared with the budget:

Condition	Message shown	Colour
Spent is greater than budget	"Budget exceeded by ₹X."	Red
Spent is equal to budget	"Daily budget reached!"	Red
Spent is 80% or more of budget	"Careful: you have used at least 80% of today's budget."	Amber
Otherwise	"You are within today's budget."	Grey

8. Implementation

The application consists of one Java class (`MainActivity.java`, about 480 lines), one XML layout, seven drawable shape files and three resource files for colours, strings and styles. The main methods are described below.

8.1 Application start-up

`onCreate()` sets the layout, opens SharedPreferences in private mode, binds the views using `findViewById()`, loads saved data and displays the dashboard. The Add Expense and Set Budget buttons are connected to their dialogs in `bindViews()`.

8.2 Adding an expense

`showAddExpenseDialog()` builds an `AlertDialog` containing four inputs: expense title, amount in rupees, a category drop-down (Spinner) and an optional note. The Save button is attached in an `OnShowListener`, so that the dialog stays open when validation fails instead of closing automatically. Two checks are made:

- The title must not be empty.
- The amount must be a valid number greater than zero.

On success, a new `Expense` is created with the current time as both its id and timestamp, added to the list, saved and the dashboard refreshed.

8.3 Converting rupees to paise

The amount typed by the user is converted safely using `BigDecimal`:

```
BigDecimal rupees = new BigDecimal(normalized);
BigDecimal paise = rupees.multiply(new BigDecimal("100"))
                         .setScale(0, RoundingMode.HALF_UP);
```

Commas are removed from the input first, zero or negative values are rejected, and invalid text returns `null`, which the dialog treats as an error.

8.4 Setting the daily budget

`showBudgetDialog()` shows the current budget and lets the user enter a new one. The same validation is applied, and the new value is stored under `daily_budget_paise`. If a stored budget is ever zero or negative, `loadBudget()` resets it to the default of ₹500.

8.5 Saving and loading data

`saveExpenses()` converts the list of expenses into a `JSONArray` and stores it as a string. `loadExpenses()` reads it back and rebuilds the list. While loading, each record is checked: it must have a non-empty id and title, an amount greater than zero and a valid timestamp, otherwise it is skipped. If the stored text cannot be parsed at all, the list is reset and a Toast message informs the user. This makes the app resistant to corrupted data.

8.6 Dashboard calculation

`refreshDashboard()` is the central method. It sorts the expenses so that the newest appears first, shows today's date, and then computes:

- **Spent today:** the sum of expenses whose timestamp falls on the current day (checked by `isToday()` using year and day of the year).
- **Available:** budget minus spent, never shown below zero.
- **Progress:** spent × 100 / budget, limited to a maximum of 100 for the progress bar.

It then updates the warning message, the category summary and the transaction list.

8.7 Category summary and transaction list

`renderCategorySummary()` adds up today's spending for each of the six categories using a `LinkedHashMap`, which keeps categories in a fixed order, and creates a row for each. `renderTransactions()` builds one card for every saved expense, showing its title, amount, category, date and time, the optional note and a Delete button. If there are no expenses, the message "No expenses yet. Add your first expense!" is displayed.

8.8 Deleting an expense

Tapping Delete opens a confirmation dialog ("Delete ‘title’?"). Only after the user confirms is the expense removed, the data saved and the dashboard refreshed. This prevents accidental loss of records.

8.9 Formatting helpers

`formatMoney()` converts paise to a display string such as ₹120.00, `formatPlainRupees()` produces a clean number for the budget input box, and `formatDateTime()` produces text like "09 Oct 2026, 12:35 PM" using the device's locale.

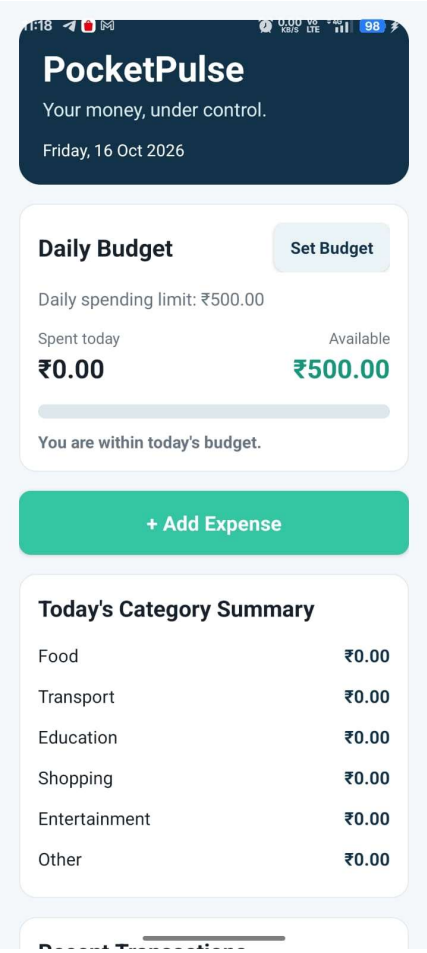
9. User Interface Design

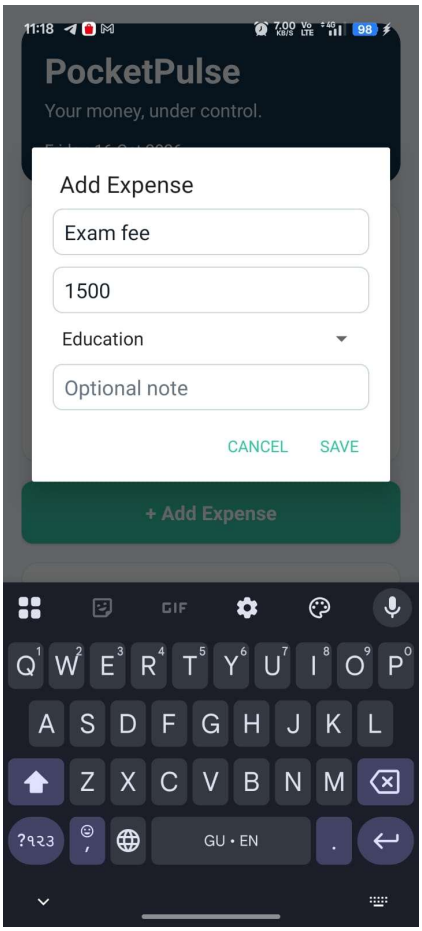
The interface is a single scrollable screen with a clean, modern look. It is built from the XML layout together with views generated in Java for the dynamic lists.

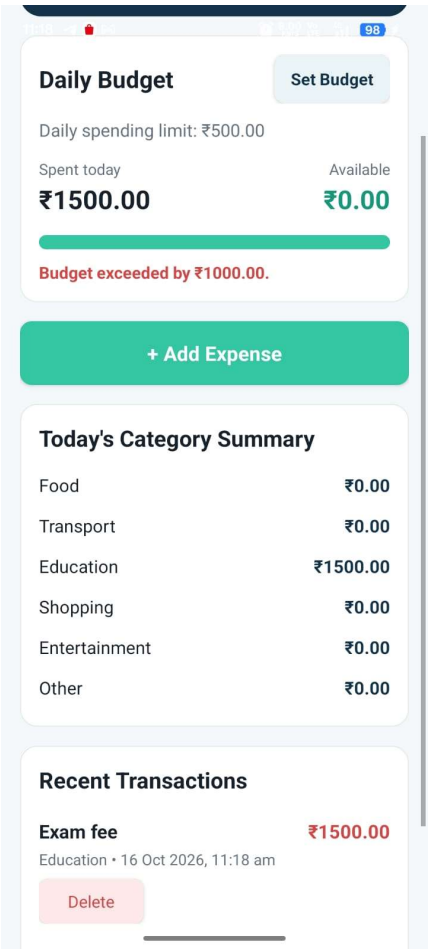
Section	Contents
Header	App name "PocketPulse", tagline "Your money, under control." and today's date, on a navy background
Daily Budget card	Set Budget button, daily limit, "Spent today" and "Available" amounts, progress bar and warning message
Add Expense button	A prominent "+ Add Expense" button
Today's Category Summary	Six rows showing spending per category
Recent Transactions	Scrollable list of all expenses with a Delete button on each

Colour palette

Name	Hex	Use
Navy	#12324A	Header and headings
Navy dark	#0B2233	Status and navigation bars
Mint	#34C6A3	Accent colour
Mint dark	#15977B	"Available" amount
Screen background	#F4F8FA	Page background
Danger	#D64545	Over-budget warnings, expense amounts, delete
Warning	#B7791F	80% budget warning
Text muted	#667581	Secondary text







The app uses the Material Light theme without an action bar, with rounded card and button backgrounds defined as drawable XML files. Colour is used to carry meaning: green and grey for safcaution, and red for exceeded.

[Insert screenshots of the app here: Home screen, Add Expense dialog, Set Budget dialog, Budget exceeded state, Delete confirmation]

10. Testing

The application was designed with input validation and error handling at every point where the user or stored data can cause a problem. The table below lists the test cases that were considered, along with the expected behaviour. [Fill in the Actual Result / Pass-Fail column after running each test on your emulator or phone.]

No.	Test case	Input	Expected result
1	Launch app for the first time	None	Dashboard opens, budget ₹500.00, spent ₹0.00, empty-state message shown
2	Add a valid expense	Title "Lunch", amount 120, category Food	Expense appears in the list; spent today becomes ₹120.00; Food total updates
3	Empty title	Title blank, amount 50	Error "Enter an expense title."; dialog stays open
4	Zero amount	Amount 0	Error "Enter a valid amount greater than zero."
5	Negative or non-numeric amount	-20 or abc	Same error; nothing saved
6	Decimal amount	99.50	Saved as 9950 paise; displayed as ₹99.50
7	Spend reaches 80%	Budget ₹500, spent ₹400	Amber warning displayed
8	Spend equals budget	Spent ₹500	"Daily budget reached!" in red; progress bar full
9	Spend exceeds budget	Spent ₹650	"Budget exceeded by ₹150.00." in red; available shows ₹0.00
10	Change budget	New budget 1000	Limit, available amount, progress and warning update immediately
11	Invalid budget	0 or blank	Error "Enter a valid budget greater than zero."
12	Delete an expense	Tap Delete, confirm	Expense removed; totals recalculated

No.	Test case	Input	Expected result
13	Cancel delete	Tap Delete, then Cancel	Expense remains
14	Persistence	Close and reopen the app	Budget and expenses are still present
15	Next-day behaviour	Open the app on a new day	Spent today resets to ₹0; older expenses stay in the list
16	Corrupted storage	Unreadable JSON	List is reset and a Toast message is shown; app does not crash

11. Limitations

- Expenses cannot be edited after saving; they can only be deleted and re-entered.
- The budget and category summary cover only the current day. There is no weekly or monthly report.
- The transaction list shows all expenses together without filtering or searching.
- There are no charts or graphs.
- Data is stored only on the device. If the app is uninstalled or the data is cleared, the records are lost, and there is no export or backup option.
- All expenses are stored as one JSON string, which is simple but not ideal for a very large number of records, because the whole list is rewritten on every change.
- The category list is fixed and cannot be customised, and only Indian Rupees are supported.
- All logic is placed in a single activity class, and automated unit tests are not included.

12. Future Enhancements

1. Edit existing expenses.
2. Weekly and monthly budgets with reports, and filters by date or category.
3. Pie and bar charts for spending analysis.
4. Migration from SharedPreferences to a Room (SQLite) database for better performance with large data.
5. Daily reminder and budget alert notifications.
6. Export to CSV or PDF and cloud backup.
7. Custom categories, income tracking and multiple currencies.
8. Dark mode and biometric app lock.
9. Refactoring into the MVVM architecture with unit and UI tests.

13. Conclusion

PocketPulse meets its goal of providing a simple, private and practical way to track daily spending. Users can record an expense in a few taps, immediately see how much of their daily budget remains, and receive clear colour-coded warnings before they overspend. The app uses only standard Android

components, works completely offline, stores money accurately as whole paise, and validates all input and stored data to avoid errors and crashes.

Developing this project gave practical experience with the Android activity lifecycle, XML and programmatic UI design, dialogs and input validation, JSON handling, local data persistence with SharedPreferences, and precise money calculation with BigDecimal. The project also provides a solid base that can be extended with the enhancements listed above.

14. References

1. Android Developers, "Documentation for app developers": https://developer.android.com/docs
2. Android Developers, "Save key-value data with SharedPreferences": https://developer.android.com/training/data-storage/shared-preferences
3. Android Developers, "Dialogs" and "Layouts" guides: https://developer.android.com/develop/ui
4. Oracle, Java SE API documentation for `java.math.BigDecimal`: https://docs.oracle.com/en/java/javase/
5. Android API reference for the `org.json` package: https://developer.android.com/reference/org/json/package-summary
6. Gradle User Manual: https://docs.gradle.org

Appendix A: Project Structure

```
pocketpulse/
├── build.gradle
├── settings.gradle
├── gradle.properties
├── gradle/wrapper/gradle-wrapper.properties
└── app/
    ├── build.gradle
    └── src/main/
        ├── AndroidManifest.xml
        ├── java/com/example/pocketpulse/
        │   └── MainActivity.java
        └── res/
            ├── layout/activity_main.xml
            ├── drawable/
            │   ├── card_background.xml
            │   ├── header_background.xml
            │   ├── edit_text_background.xml
            │   ├── primary_button_background.xml
            │   ├── secondary_button_background.xml
            │   ├── delete_button_background.xml
            │   └── progress_drawable.xml
            └── values/
                ├── colors.xml
                ├── strings.xml
                └── styles.xml
```

Key configuration (app/build.gradle)

Setting	Value
namespace / applicationId	com.example.pocketpulse
compileSdk / targetSdk	36
minSdk	23
versionCode / versionName	1 / 1.0